SiR^2ENA

Simple Register-Register Extensible Neo-Architecture

Instruction Set Architecture

1.0

October 25, 2026

Contents

1 Preface **3**

2 Glossary **3**

3 Design **3**
- 3.1 Instruction types . . . 3
  - 3.1.1 Register type (R-type) . . . 3
  - 3.1.2 Immediate type (I-type) . . . 3
  - 3.1.3 Immediate jump type (J-type) . . . 3
- 3.2 Instruction encoding . . . 3
  - 3.2.1 R-type . . . 3
  - 3.2.2 I-type . . . 3
  - 3.2.3 J-type . . . 3

4 Core set (CORE32) **3**
- 4.1 Miscellaneous . . . 3
  - 4.1.1 nop - no operation . . . 3
  - 4.1.2 cpuid - CPU identification . . . 4
  - 4.1.3 cpuft - CPU feature . . . 4
- 4.2 Load/Store operations . . . 4
  - 4.2.1 ldw - load word . . . 4
  - 4.2.2 ldhz - load half-word zero-extended . . . 4
  - 4.2.3 ldhs - load half-word sign-extended . . . 5
  - 4.2.4 ldbz - load byte zero-extended . . . 5
  - 4.2.5 ldbs - load byte sign-extended . . . 5
  - 4.2.6 lvi - load sign-extended value immediate . . . 5
  - 4.2.7 lli - load lower half-word immediate . . . 6
  - 4.2.8 lui - load upper half-word immediate . . . 6
  - 4.2.9 stw - store word . . . 6
  - 4.2.10 sth - store half-word . . . 6
  - 4.2.11 stb - store byte . . . 7
- 4.3 Register operations . . . 7
  - 4.3.1 mov - move . . . 7
- 4.4 Control flow . . . 7

4.4.1 ja - jump absolute . . . 7
4.4.2 jae - jump absolute if equal . . . 7
4.4.3 jane - jump absolute if not equal . . . 8
4.4.4 jalt - jump absolute if less than . . . 8
4.4.5 jage - jump absolute if greater or equal . . . 8
4.4.6 jr - jump relative . . . 9
4.4.7 jre - jump relative if equal . . . 9
4.4.8 jrne - jump relative if not equal . . . 9
4.4.9 jrlt - jump relative if less than . . . 10
4.4.10 jrge - jump relative if greater or equal . . . 10
4.4.11 calla - call procedure absolute . . . 10
4.4.12 callr - call procedure relative . . . 11
4.4.13 ret - return from procedure . . . 11
4.5 Mathematical operations . . . 11
4.5.1 cmpu - compare values (unsigned) . . . 11
4.5.2 cmps - compare values (signed) . . . 12
4.5.3 add - integer addition . . . 12
4.5.4 sub - integer subtraction . . . 13
4.5.5 addi - integer addition sign-extended immediate . . . 13
4.5.6 mul - integer multiplication . . . 13
4.6 Bitwise operations . . . 13
4.6.1 and - bitwise AND . . . 13
4.6.2 or - bitwise OR . . . 14
4.6.3 xor - bitwise XOR . . . 14
4.6.4 shl - logical shift left . . . 14
4.6.5 shr - logical shift right . . . 14
4.6.6 shsr - arithmetic (signed) shift right . . . 15
4.6.7 shli - logical shift left immediate . . . 15
4.6.8 shri - logical shift right immediate . . . 15
4.6.9 shsri - arithmetic (signed) shift right immediate . . . 15
4.7 Interrupt control . . . 16
4.7.1 int - raise software interrupt . . . 16
4.7.2 inti - raise software interrupt immediate . . . 16
4.7.3 intm - mask interrupt . . . 16
4.7.4 reti - return from interrupt handler . . . 16
4.8 Memory operations . . . 17
4.8.1 mcw - copy word . . . 17
4.8.2 mch - copy half-word . . . 17
4.8.3 mcb - copy byte . . . 17

1 Preface

2 Glossary

3 Design

3.1 Instruction types

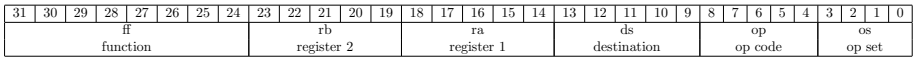
3.1.1 Register type (R-type)

3.1.2 Immediate type (I-type)

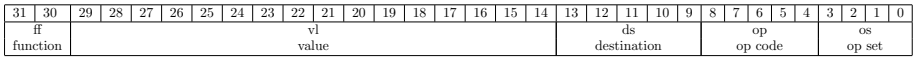
3.1.3 Immediate jump type (J-type)

3.2 Instruction encoding

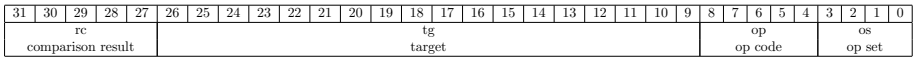
3.2.1 R-type



3.2.2 I-type



3.2.3 J-type



4 Core set (CORE32)

CPUID bit 0, required

4.1 Miscellaneous

4.1.1 nop - no operation

os	op	ds	ra	rb	ff
0x00	0x00	–	–	–	–

Performs no operation. The number of cycles consumed by this instruction is implementation-dependent and not guaranteed.

4.1.2 cpuid - CPU identification

os	op	ds	vl	ff
0x00	0x01	result	–	0x00

Writes the CPUID structure into *result*. See CPU: CPUID

4.1.3 cpuft - CPU feature

os	op	ds	vl	ff
0x00	0x01	result	feature	0x01

Writes information about a specific CPU feature into *result*. A *feature* value of 0 returns the available features (See CPU:13.1 [Feature ID table]). If information about a non-existing feature is requested, the CPU raises a **#IOP**.

Implementation

```
1 // implementation-specific
```

4.2 Load/Store operations

4.2.1 ldw - load word

os	op	ds	ra	rb	ff
0x00	0x02	destination	&source (4-aligned)	–	0x00

Loads a single word from memory at *source* into *destination*. Paging applies.

NOTE: *source* **must** be word-aligned. If it isn't, the MMU will raise a **#MAE**

Implementation

```
1 destination = mread32(tavp(source, R))
```

4.2.2 ldhz - load half-word zero-extended

os	op	ds	ra	rb	ff
0x00	0x02	destination	&source (2-aligned)	–	0x01

Loads a zero-extended half-word from memory at *source* into *destination*. Paging applies.

NOTE: *source* **must** be half word-aligned. If it isn't, the MMU will raise a **#MAE**

Implementation

```
1 destination = zeroe16(mread16(tavp(source, R)))
```

4.2.3 ldhs - load half-word sign-extended

os	op	ds	ra	rb	ff
0x00	0x02	destination	&source (2-aligned)	–	0x02

Loads a sign-extended half-word from memory at *source* into *destination*. Paging applies.

NOTE: *source* **must** be half word-aligned. If it isn't, the MMU will raise a **#MAE**

Implementation

```
1 destination = signe16(mread16(tavp(source, R)))
```

4.2.4 ldbz - load byte zero-extended

os	op	ds	ra	rb	ff
0x00	0x02	destination	&source	–	0x03

Loads a zero-extended byte from memory at *source* into *destination* . Paging applies.

Implementation

```
1 destination = zeroe8(mread8(tavp(source, R)))
```

4.2.5 ldbs - load byte sign-extended

os	op	ds	ra	rb	ff
0x00	0x02	destination	&source	–	0x04

Loads a sign-extended byte from memory at *source* into *destination* . Paging applies.

Implementation

```
1 destination = signe8(mread8(tavp(source, R)))
```

4.2.6 lvi - load sign-extended value immediate

os	op	ds	vl	ff
0x00	0x03	destination	value	–

Sign-extends and loads *value* into *destination*

Implementation

```
1 destination = signe16(value)
```

4.2.7 lli - load lower half-word immediate

os	op	ds	vl	ff
0x00	0x04	destination	value	0x00

Loads *value* into the lower half-word of *destination*

Implementation

```
1 destination = (destination & 0xFFFF0000) | value
```

4.2.8 lui - load upper half-word immediate

os	op	ds	vl	ff
0x00	0x04	destination	value	0x01

Loads *value* into the upper half-word of *destination*

Implementation

```
1 destination = (destination & 0x0000FFFF) | (value <<
    16)
```

4.2.9 stw - store word

os	op	ds	ra	rb	ff
0x00	0x05	&destination (4-aligned)	source	–	0x00

Writes a single word into memory at *destination* from *source*. Paging applies.

NOTE: *destination* **must** be word-aligned. If it isn't, the MMU will raise a **#MAE**

Implementation

```
1 mwrite32(tavp(destination, W), source)
```

4.2.10 sth - store half-word

os	op	ds	ra	rb	ff
0x00	0x05	&destination (2-aligned)	source	–	0x01

Writes a half-word into memory at *destination* from *source*. Paging applies.

NOTE: *destination* **must** be half word-aligned. If it isn't, the MMU will raise a **#MAE**

Implementation

```
1 mwrite16(tavp(destination, W), source & 0xFFFF)
```

4.2.11 stb - store byte

os	op	ds	ra	rb	ff
0x00	0x05	&destination	source	–	0x02

Writes a byte into memory at *destination* from *source*. Paging applies.

Implementation

```
1 mwrite8(tavp(destination, W), source & 0xFF)
```

4.3 Register operations

4.3.1 mov - move

os	op	ds	ra	rb	ff
0x00	0x06	destination	source	–	–

Copies the value of *source* into *destination*

Implementation

```
1 destination = source
```

4.4 Control flow

4.4.1 ja - jump absolute

os	op	ds	ra	rb	ff
0x00	0x07	&target	–	–	–

Sets the *instruction pointer* to the value of *target*, which marks *target* as the immediate next instruction to be executed by the CPU.

Implementation

```
1 rip = target
```

4.4.2 jae - jump absolute if equal

os	op	ds	ra	rb	ff
0x00	0x08	&target	cmp	–	0x00

Sets the *instruction pointer* to the value of *target* if the comparison result *cmp* is **EQUAL**, which marks *target* as the immediate next instruction to be executed by the CPU.

Implementation

```
1 if (cmp & 0b0001) != 0 {
2     rip = target
3 }
```

4.4.3 jane - jump absolute if not equal

os	op	ds	ra	rb	ff
0x00	0x08	&target	cmp	–	0x01

Sets the *instruction pointer* to the value of *target* if the comparison result *cmp* is **NOT EQUAL**, which marks *target* as the immediate next instruction to be executed by the CPU.

Implementation

```
1 if (cmp & 0b0010) != 0 {
2     rip = target
3 }
```

4.4.4 jalt - jump absolute if less than

os	op	ds	ra	rb	ff
0x00	0x08	&target	cmp	–	0x02

Sets the *instruction pointer* to the value of *target* if the comparison result *cmp* is **LESS THAN**, which marks *target* as the immediate next instruction to be executed by the CPU.

Implementation

```
1 if (cmp & 0b0100) != 0 {
2     rip = target
3 }
```

4.4.5 jage - jump absolute if greater or equal

os	op	ds	ra	rb	ff
0x00	0x08	&target	cmp	–	0x04

Sets the *instruction pointer* to the value of *target* if the comparison result *cmp* is **GREATER THAN** or **EQUAL**, which marks *target* as the immediate next instruction to be executed by the CPU.

Implementation

```
1 if (cmp & 0b1000) != 0 || (cmp & 0b0001) != 0 {
2     rip = target
3 }
```

4.4.6 jr - jump relative

os	op	tg	rc
0x00	0x09	target	–

Sets the *instruction pointer* to sign-extended *target* relative to the current value of the *instruction pointer*, which marks *rip + target* as the immediate next instruction to be executed by the CPU.

Implementation

```
1 rip = rip + signe18(target)
```

4.4.7 jre - jump relative if equal

os	op	tg	rc
0x00	0x0A	target	cmp

Sets the *instruction pointer* to sign-extended *target* relative to the current value of the *instruction pointer* if the comparison result *cmp* is **EQUAL**, which marks *rip + target* as the immediate next instruction to be executed by the CPU.

Implementation

```
1 if (cmp & 0b0001) != 0 {
2     rip = rip + signe18(target)
3 }
```

4.4.8 jrne - jump relative if not equal

os	op	tg	rc
0x00	0x0B	target	cmp

Sets the *instruction pointer* to sign-extended *target* relative to the current value of the *instruction pointer* if the comparison result *cmp* is **NOT EQUAL**, which marks *rip + target* as the immediate next instruction to be executed by the CPU.

Implementation

```
1 if (cmp & 0b0010) != 0 {
2     rip = rip + signe18(target)
3 }
```

4.4.9 jrlt - jump relative if less than

os	op	tg	rc
0x00	0x0C	target	cmp

Sets the *instruction pointer* to sign-extended *target* relative to the current value of the *instruction pointer* if the comparison result *cmp* is **LESS THAN**, which marks *rip + target* as the immediate next instruction to be executed by the CPU.

Implementation

```
1 if (cmp & 0b0100) != 0 {
2     rip = rip + signe18(target)
3 }
```

4.4.10 jrge - jump relative if greater or equal

os	op	tg	rc
0x00	0x0E	target	cmp

Sets the *instruction pointer* to sign-extended *target* relative to the current value of the *instruction pointer* if the comparison result *cmp* is **GREATER THAN** or **EQUAL**, which marks *rip + target* as the immediate next instruction to be executed by the CPU.

Implementation

```
1 if (cmp & 0b1000) != 0 || (cmp & 0b0001) != 0 {
2     rip = rip + signe18(target)
3 }
```

4.4.11 calla - call procedure absolute

os	op	ds	ra	rb	ff
0x00	0x10	&target	–	–	–

Sets the *procedure tail pointer* to the value of the *instruction pointer* and then sets the *instruction pointer* to the value of *target*, which marks *target* as the immediate next instruction to be executed by the CPU.

Implementation

```
1 rtp = rip + 4
2 rip = target
```

4.4.12 callr - call procedure relative

os	op	tg	rc
0x00	0x11	target	–

Sets the *procedure tail pointer* to the value of the *instruction pointer* and then sets the *instruction pointer* to the value of sign-extended *target* relative to the current value of the *instruction pointer*, which marks *rip + target* as the immediate next instruction to be executed by the CPU.

Implementation

```
1 rtp = rip + 4
2 rip = rip + signe18(target)
```

4.4.13 ret - return from procedure

os	op	ds	ra	rb	ff
0x00	0x12	–	–	–	–

Sets the *instruction pointer* to the value of the *procedure tail pointer*, which marks *rtp* as the immediate next instruction to be executed by the CPU.

Implementation

```
1 rip = rtp
```

4.5 Mathematical operations

NOTE: These instructions may all modify the `rcf` register in accordance to the specific operation. For more information see ALU operations.

4.5.1 cmpu - compare values (unsigned)

os	op	ds	ra	rb	ff
0x00	0x13	result	a	b	0x00

Compares the unsigned values of *a* and *b* and stores the comparison result in *result*. It does not modify *rcf*.

The comparison result is a 4-bit flag value and maps to the following results:

bit	mnemonic	description
0	EQUAL	if set, a and b are equal
1	NOT EQUAL	if set, a and b are not equal
2	LESS THAN	if set, a is less than b
3	GREATER THAN	if set, a is greater than b

Implementation

```
result = 0

if a == b {
    result = result | (1 << 0)
} else {
    result = result | (1 << 1)

    if a < b {
        result = result | (1 << 2)
    }

    if a > b {
        result = result | (1 << 3)
    }
}
```

4.5.2 cmps - compare values (signed)

os	op	ds	ra	rb	ff
0x00	0x13	result	a	b	0x01

Signed version of *cmp*, comparing signed values.

Implementation

```
-- see cmpu
```

4.5.3 add - integer addition

os	op	ds	ra	rb	ff
0x00	0x14	result	a	b	0x00

Adds the values of a and b together and stores the result in *result*

Implementation

```
result = iadd32c(a, b)
```

4.5.4 sub - integer subtraction

os	op	ds	ra	rb	ff
0x00	0x14	result	a	b	0x01

Subtracts the value of *b* from the value of *a* and stores the result in *result*

Implementation

```
1 result = isub32(a, b)
```

4.5.5 addi - integer addition sign-extended immediate

os	op	ds	vl	ff
0x00	0x15	a	b	–

Adds the value of sign-extended *b* and the value of *a* together, storing the result in *a*

Implementation

```
1 a = iadd32c(a, signe16(b))
```

4.5.6 mul - integer multiplication

os	op	ds	ra	rb	ff
0x00	0x16	result	a	b	0x00

Multiplies the values of *a* and *b* together and stores the result in *result*

Implementation

```
1 result = imul32(a, b)
```

4.6 Bitwise operations

4.6.1 and - bitwise AND

os	op	ds	ra	rb	ff
0x00	0x17	result	a	b	0x00

Performs a bitwise AND operation on the values of *a* and *b* and stores the result in *result*

Implementation

```
1 result = and32(a, b)
```

4.6.2 or - bitwise OR

os	op	ds	ra	rb	ff
0x00	0x17	result	a	b	0x01

Performs a bitwise OR operation on the values of a and b and stores the result in *result*

Implementation

```
1 result = or32(a, b)
```

4.6.3 xor - bitwise XOR

os	op	ds	ra	rb	ff
0x00	0x17	result	a	b	0x02

Performs a bitwise XOR operation on the values of a and b and stores the result in *result*

Implementation

```
1 result = xor32(a, b)
```

4.6.4 shl - logical shift left

os	op	ds	ra	rb	ff
0x00	0x18	result	a	n	0x00

Shifts the value of a left by n places and stores the result in *result*

Implementation

```
1 result = lshl32(a, n)
```

4.6.5 shr - logical shift right

os	op	ds	ra	rb	ff
0x00	0x18	result	a	n	0x01

Shifts the value of a right by n places and stores the result in *result*

Implementation

```
1 result = lshr32(a, n)
```

4.6.6 shsr - arithmetic (signed) shift right

os	op	ds	ra	rb	ff
0x00	0x18	result	a	n	0x02

Shifts the value of *a* right by *n* places and stores the result in *result*, sign-replicating it

Implementation

```
1 result = ashr32(a, n)
```

4.6.7 shli - logical shift left immediate

os	op	ds	vl	ff
0x00	0x19	a	n	0x00

Shifts the value of *a* in-place left by *n* places

Implementation

```
1 a = lshl32(a, signe16(n))
```

4.6.8 shri - logical shift right immediate

os	op	ds	vl	ff
0x00	0x19	a	n	0x01

Shifts the value of *a* in-place right by *n* places

Implementation

```
1 a = lshr32(a, signe16(n))
```

4.6.9 shsri - arithmetic (signed) shift right immediate

os	op	ds	vl	ff
0x00	0x19	a	n	0x02

Shifts the value of *a* in-place right by *n* places, sign-replicating the result

Implementation

```
1 a = ashr32(a, signe16(n))
```

4.7 Interrupt control

4.7.1 int - raise software interrupt

os	op	ds	ra	rb	ff
0x00	0x1A	id	–	–	–

Raises a software interrupt. Interrupt arguments are read from *rfi0* and *rfi1*.

NOTE: PL 0 callers may only raise interrupts in the *user-defined* range. Trying to raise a non-user interrupt from PL 0 will result in a **#GPE**.

4.7.2 inti - raise software interrupt immediate

os	op	ds	vl	ff
0x00	0x1B	–	id	–

Raises a software interrupt. Interrupt arguments are read from *rfi0* and *rfi1*.

NOTE: PL 0 callers may only raise interrupts in the *user-defined* range. Trying to raise a non-user interrupt from PL 0 will result in a **#GPE**.

4.7.3 intm - mask interrupt

os	op	ds	vl	ff
0x00	0x1C	id	enable	–

If *enable* is zero masks an interrupt, telling the CPU not to handle it through a Interrupt Routine Handler. Otherwise, (re-)enables it.

NOTE: This instruction is restricted to PL 1. Calls from PL 0 will raise a **#GPE**.

Implementation

```
1 if enable {
2     rrap[id * 0xC] = rrap[id * 0xC] | 1
3 } else {
4     rrap[id * 0xC] = rrap[id * 0xC] & ~1
5 }
```

4.7.4 reti - return from interrupt handler

os	op	ds	ra	rb	ff
0x00	0x1D	–	–	–	–

Sets the *register frame pointer* to the value of the *last frame pointer* and adds 4 to the *instruction pointer* to proceed to the next instruction. This instruction should only be called inside of a Interrupt Routine Handler.

Implementation

```
1 rfp = rlfp
2 rip = rip + 4
```

4.8 Memory operations

4.8.1 mcw - copy word

os	op	ds	ra	rb	ff
0x00	0x1E	&destination (4-aligned)	&source (4-aligned)	–	0x00

Copies a single word from memory at *source* directly into memory at *destination*. Paging applies.

NOTE: *source* and *destination* **must** be word-aligned. If it isn't, the MMU will raise a **#MAE**

Implementation

```
1 mwrite32(tavp(destination, W), mread32(tavp(source, R)
    ))
```

4.8.2 mch - copy half-word

os	op	ds	ra	rb	ff
0x00	0x1E	&destination (2-aligned)	&source (2-aligned)	–	0x01

Copies a single half-word from memory at *source* directly into memory at *destination*. Paging applies.

NOTE: *source* and *destination* **must** be half-word-aligned. If it isn't, the MMU will raise a **#MAE**

Implementation

```
1 mwrite16(tavp(destination, W), mread16(tavp(source, R)
    ))
```

4.8.3 mcb - copy byte

os	op	ds	ra	rb	ff
0x00	0x1E	&destination	&source	–	0x02

Copies a single byte from memory at *source* directly into memory at *destination*. Paging applies.

Implementation

```
1  mwrite8(tavp(destination, W), mread8(tavp(source, R)))
```